Stappenplan Bouwcontrole — V3

STAPPENPLAN V3 · Bouwcontrole · tot de demo van 17 januari 2027 · 8 november 2026

Dit stappenplan brengt Bouwcontrole tot een demo die je op het werk toont: twee voorbeeldcomputers, gecontroleerd door je eigen API, met hun conflicten in kleur en tekst op het scherm. Het volgt de lessen die je gezien hebt, en het ERD van je databaseproject.

Er staat geen code in. Per fase staat wat je doet en waarom, nooit hoe. Lees de les die bij een fase staat eerst: de fase gaat ervan uit dat je ze kent.

Wat er veranderde op 8 november

Wat	Vroeger	Nu
De volgorde	de controle pas na les 09	de controle meteen na les 06; les 07 tot 09 na de demo
Tijdelijke opslag	een JSON-bestand	geen: meteen de database
Je gegevens ontwerpen	zelf afleiden uit je lijst	je ERD, het deel over de onderdelen
De omvang tot de demo	alle types	zes types, twee voorbeeldcomputers, ongeveer tien regels
Het scherm	pas na alle lessen	de alpha, gekoppeld aan je API, toont het resultaat van je echte controle; het scherm in je eigen project volgt in fase 14
Wat een conflict toont	een ernst en een tekst	de ernst, de onderdelen die samen het conflict geven, de uitleg en een mogelijke oplossing
Je database vullen	verspreid over de fases	een eigen hoofdstuk, stap voor stap

Beslist op 8 november 2026, en vastgelegd in de projectafspraken, §9.

De demo

Op 17 januari is dit klaar:

Wat je toont	Waarmee	Wat het bewijst
De controle	Scalar: je stuurt de bouworder van elke voorbeeldcomputer in, en krijgt de conflicten terug met hun ernst en uitleg	de controle werkt echt, op echte onderdelen
De gegevens	pgAdmin: de onderdelen en de regels in je database	alles staat in een echte database
Het scherm	de alpha, als venster op je API: je scant de codes, de alpha vraagt de controle aan je API, en toont wat terugkomt	dat de werkvloer je echte controle kan lezen

Wat het scherm toont:

- per onderdeel een kleur: groen is in orde, oranje kan technisch maar heeft een gevolg, rood kan niet;
- per conflict, dus per verband tussen onderdelen: welke onderdelen het zijn, waarom het fout loopt, en een mogelijke oplossing.

Het scherm controleert zelf niets. Het toont wat je API teruggeeft. Klopt je controle, dan klopt het scherm, ook voor een computer die je niet vooraf koos. Testen doe je dus op je API: in fase 7 tegen je verwachte uitkomsten. In fase 8 kijk je of het scherm hetzelfde toont als Scalar.

Waarom de alpha, en niet je eigen project. Een scherm in je eigen project is .NET Development, les 09 en 10, en kost 10 tot 15 uur extra. De alpha is een los project waarin ik de code mag geven: de uitzondering van oktober, op 8 november uitgebreid tot de koppeling aan je API. Je API en je controle blijven volledig jouw werk. Na de demo bouw je het scherm zelf, in fase 14.

De zes types: processor, moederbord, werkgeheugen, koeling, behuizing en ventilator. Videokaart, voeding en opslaggeheugen komen na de demo.

Waar je naartoe werkt

Op de dag van de demo ziet je webproject er zo uit. De mappen komen er fase per fase bij; maak ze niet vooraf aan.

Map of bestand	Wat erin staat	Vanaf fase
Controllers/	de endpoints: onderdelen opzoeken, een bouworder controleren	1
Models/	Onderdeel, de zes types, hun enumeraties en de conflictregels	2
Data/	je DbContext	3
Migrations/	door Entity Framework gemaakt	3
docker-compose.yml	PostgreSQL en pgAdmin	3
Repositories/	de interfaces en hun implementaties	4
Services/	de controle, ook de engine genoemd	7

De lagen. Een verzoek komt binnen in een controller. Die vraagt de gegevens aan een repository, en geeft ze aan de service die controleert. Het antwoord gaat langs dezelfde controller terug. De service kent geen database en geen HTTP: zo blijft ze staan als je na de demo verbouwt, en kan je ze later testen.

DTOs/, Configuration/ en Views/ komen pas na de demo.

De fases in één oogopslag

Fase	Les eerst	Wat het oplevert	Uren
0 · Opruimen	—	je map en GitHub op orde	1–2
1 · Het project draait	PA 01	het project bouwt; één eigen endpoint in Scalar	1–2
2 · Afstemmen en kiezen	je ERD	je twee computers, je demoregels en de verwachte uitkomst; je modellen volgen je ERD	4–6
3 · De database	PA 04	Docker, PostgreSQL, DbContext en de eerste migratie, voor één type	4–6
4 · De repository	PA 03 en 05	asynchroon opzoeken, ook op Kraken-Id	3–5
5 · De zes types	PA 06	alle types, de enumeraties en de overerving	7–12
6 · Regels en startgegevens	PA 06	je demoregels, en de onderdelen van je twee computers	6–10
7 · De controle	PA 03, 05 en 06	de controle als service, en het endpoint dat een bouworder controleert	8–12
8 · De demo	—	de alpha gekoppeld aan je API, het script, de generale repetitie	5–8
Tot de demo			**39–63**

De kalender, de mijlpalen en de taken staan in de takenlijst V3. Rechtstreeks naar:

- Je database vullen · stap voor stap
- Fase 0 · Opruimen
- Fase 1 · Het project draait
- Fase 2 · Afstemmen en kiezen
- Fase 3 · De database
- Fase 4 · De repository
- Fase 5 · De zes types
- Fase 6 · Regels en startgegevens
- Fase 7 · De controle
- Fase 8 · De demo
- Deel 2 · Na de demo

Je database vullen

Je database krijgt haar gegevens via **startgegevens**: rijen die je in je DbContext meegeeft met HasData, en die met je migraties in de tabellen komen. Dat is de manier van Programming Advanced les 06. Je hebt er geen nieuwe les voor nodig, en geen apart programma.

Waar het uitgelegd staat.

- Programming Advanced les 06.
- Leerboek C#, hoofdstuk 39: *Stap 4: de startgegevens verhuizen* en *Stap 5: OnModelCreating opsplitsen*, met de fouten onder *Als een stap ontbreekt*.
- Leerboek C#, hoofdstuk 31: *Startgegevens met HasData*.
- Oefenen: 06_01 en 06_04 geven startgegevens aan twee tabellen met een relatie ertussen. Maak er één voor je aan fase 3 begint.

Waarom zo, en niet met de hand. Typ je gegevens in pgAdmin, dan zijn ze weg zodra je je database opnieuw opbouwt, en dat gebeurt misschien in fase 5. Startgegevens komen bij elke Update-Database terug. Ze staan ook in je code, en dus op GitHub.

De vijf stappen

Stap	Wat je doet	Wanneer
1. Verzamelen	Een gegevensblad, in Excel of Word: per type een tabel, met één rij per onderdeel en één kolom per eigenschap die je demoregels nodig hebben, plus Kraken-Id, Merk en Naam. De waarden haal je van de productpagina's en uit je systemen op het werk.	fase 2
2. Nummeren	Geef elk onderdeel een Id. Een Id is uniek over alle types heen. Geef elk type een eigen reeks, bijvoorbeeld een eigen honderdtal, dan botsen ze nooit.	fase 2
3. Overzetten	In de methode met je startgegevens, die les 06 SeedData noemt: per type de rijen van je blad. Een relatie geef je mee met de vreemde sleutel, nooit met een object. Een enum met zijn waarde. Bij overerving geef je de rijen mee bij het type zelf, niet bij Onderdeel: van een abstracte klasse bestaat geen object.	fase 3, 5 en 6
4. Migratie	Een migratie met een naam die zegt wat erbij komt, en daarna de database bijwerken.	na elke stap 3
5. Controleren	In pgAdmin: staan de rijen erin? In Scalar: vind je een onderdeel op zijn Kraken-Id, met al zijn gegevens?	na elke stap 4

Wat er wanneer in komt

Fase	Startgegevens
3 · De database	twee of drie onderdelen van je eerste type
5 · De zes types	een paar onderdelen van elk type
6 · Regels en startgegevens	alle onderdelen van je twee computers, hun alternatieven, en je demoregels: samen ongeveer 20 tot 30 onderdelen en 10 regels

Als het misloopt

De meldingen die je het vaakst ziet. Hoofdstuk 39 van je leerboek legt ze uit.

Je ziet	Wat er mis is
Add-Migration: *a default value was provided for the required property 'Id'*	een rij zonder Id. Elke rij krijgt zelf een Id.
Add-Migration: *the seed entity … has the navigation … set*	je gaf een object mee in plaats van de vreemde sleutel.
Update-Database: *violates foreign key constraint*	een rij verwijst naar een Id dat niet bestaat.
Update-Database: *duplicate key value*	twee rijen met hetzelfde Id, of gegevens die er al stonden.
De eerste POST na je startgegevens: een 500 met *duplicate key*	de teller van PostgreSQL weet niet welke Id's je startgegevens al gebruiken. Je demo voegt niets toe via je API: dit speelt pas na de demo.

Een startgegeven aanpassen of bijzetten is een nieuwe migratie, net als een kolom. Enkel in fase 5, als je keuze voor de overerving verandert, mag je je migraties weggooien en je database opnieuw opbouwen.

Deel 1 — Tot de demo

Fase 0 · Opruimen

Lees eerst: niets. **Uit je ERD:** niets. **Uren:** 1 tot 2.

Waarom nu. Je werkmap heeft wijzigingen die nog niet op GitHub staan. Het weer-voorbeeld is half weg, en het alpha-project uit je repo is verwijderd maar nog niet bewaard. Begin met een stand die je altijd kan terughalen.

Wat je doet.

- Bewaar je huidige stand op GitHub, met een boodschap die zegt wat het is.
- Draai de projectreferentie om. Nu verwijst je webproject naar je testproject; het hoort omgekeerd. Anders ziet je testproject je klassen niet, en sleept je API de testpakketten mee.
- Haal Scalar uit het testproject. Een testproject heeft geen API-pagina nodig.

Wat je niet doet. Nog geen modellen aanpassen. Dat is fase 2.

Klaar wanneer Git geen openstaande wijzigingen meer meldt, en GitHub dezelfde stand toont als je laptop.

Valkuil. Git ziet een verwijderd bestand ook als een wijziging. Controleer na je commit op GitHub dat het oude alpha-project, het weer-voorbeeld en de opgeruimde mappen er ook echt niet meer staan.

Fase 1 · Het project draait

Lees eerst: Programming Advanced les 01. **Uit je ERD:** niets. **Uren:** 1 tot 2.

Waarom nu. Bij elke fout die later komt, wil je weten dat ze aan die fout ligt, en niet aan de opzet.

Wat je doet.

- Laat het project weer bouwen. Nu lukt dat niet: de lijst in ProductController is niet afgesloten, en hij maakt een moederbord aan terwijl Moederbord abstract is. Die lijst verdwijnt in fase 3 toch.
- Maak één eigen endpoint dat iets eenvoudigs teruggeeft.

Wat je niet doet. Geen database, geen modellen.

Klaar wanneer je je endpoint in Scalar aanklikt en antwoord krijgt.

Valkuil. Oudere handleidingen laten je Swashbuckle installeren. .NET 10 maakt OpenAPI zelf, en je cursus gebruikt Scalar.

Fase 2 · Afstemmen op je ERD, en je demo kiezen

Lees eerst: je ERD V2, je conflictanalyse, en je eigen incompatibiliteitenlijst. **Uren:** 4 tot 6.

Uit je ERD: Onderdeel, en de types CPU, Moederbord, Werkgeheugen, Koeling, Behuizing en Ventilator, met hun enumeraties.

Waarom nu. Hier beslis je wat de demo moet kunnen. Alles daarna werkt naar die twee computers toe. Een model zonder die beslissing groeit tot alles wat op een productpagina staat.

Wat je doet.

1. **Kies je twee voorbeeldcomputers:** bouworders die op het werk gebouwd worden. Kies ze zo dat ze samen tonen: een conflict waardoor de bouw niet kan, een waarschuwing, een conflict met ventilatoren, en onderdelen die gewoon in orde zijn.
2. **Noteer per computer de onderdelen**, met hun Kraken-Id, op je gegevensblad: zie *Je database vullen*, stap 1 en 2.
3. **Kies je demoregels:** de regels uit je analyse of je eigen lijst die op deze twee computers vuren, aangevuld tot ongeveer tien. Noteer per regel de ernst, de uitleg die de werkvloer moet lezen, en een mogelijke oplossing. Neem er minstens één uit je eigen lijst van Alternate: die herkennen je collega's.
4. **Schrijf per computer de verwachte uitkomst op:** welke regels vuren, tussen welke onderdelen, met welke ernst, en welk onderdeel welke kleur krijgt. Dat is je toets in fase 7 en je script in fase 8.
5. **Kies per type de eigenschappen:** enkel wat een van je demoregels nodig heeft. De rest uit je ERD komt erbij met een migratie, wanneer een regel ze vraagt.
6. **Breng je modellen in lijn met je ERD:** dezelfde namen voor de types en hun eigenschappen, in het Nederlands. Onderdeel krijgt de vier gedeelde eigenschappen uit het hoofdmodel. De types die niet in de demo zitten, zet je buiten je project tot na de demo.
7. **Beslis hoe je de overerving naar tabellen vertaalt.** Dat is je special topic, en de beslissing valt nu, omdat je eerste migratie in fase 3 Onderdeel al bevat. Het ontwerp zet de afweging op een rij in sectie 5. Zoek in Entity Framework Core op wat TPH, TPT en TPC betekenen.
8. **Beslis of de artikelcode naast het Kraken-Id meegaat** (ontwerp, open vraag 1). Je ERD heeft de artikelcode; je app scant op het Kraken-Id.

Wat je niet doet.

- Je ERD aanpassen voor je app. Tot de beoordeling werk je in één richting: van je ERD naar je app.
- Klant-, order- of medewerkergegevens meenemen. Die horen bij het schoolproject, niet bij een controle.

Klaar wanneer je één blad hebt met je twee computers en hun onderdelen, je demoregels met ernst, uitleg en oplossing, de verwachte uitkomst per computer, en per type de eigenschappen. Je modellen dragen de namen uit je ERD, en je project bouwt.

Valkuil. De gegevens van je onderdelen opzoeken kost meer tijd dan je denkt. Begin er in deze fase al aan, ook al heb je ze pas in fase 6 nodig.

Fase 3 · De database

Lees eerst: Programming Advanced les 04; leerboek C#, hoofdstuk 31 en 37. **Uren:** 4 tot 6.

Uit je ERD: Onderdeel en één type, met je keuze voor de overerving.

Waarom nu. Vanaf hier bestaan je gegevens buiten je programma. Het is ook het moment waarop fouten duurder worden: lees de les grondig.

Wat je doet.

- Docker Desktop, en een compose-bestand dat PostgreSQL en pgAdmin start.
- De packages van Entity Framework, de connectiestring in appsettings.json, en een DbContext met Onderdeel en één type. Neem het type dat in de meeste demoregels voorkomt.
- Dat type vastleggen met de Fluent API, zoals in les 04.
- Twee of drie onderdelen van dat type als startgegevens: *Je database vullen*, stap 3 tot 5.
- De eerste migratie, en de database bijwerken.

Wat je niet doet. Nog geen repository, nog geen andere types.

Klaar wanneer je je tabel en je startgegevens in pgAdmin ziet.

Valkuil.

- Docker moet draaien voor je de database bijwerkt. Anders krijg je een fout die niets met je code te maken heeft.
- Pas je database nooit met de hand aan. Altijd via een migratie.
- Werkt Docker niet op je laptop, dan noemt de les een uitwijk met een lokale database. Verlies hier geen dagen.

Fase 4 · De repository op de database

Lees eerst: Programming Advanced les 05, en les 03 voor de repository; leerboek 29, 36 en 38. **Uren:** 3 tot 5.

Uit je ERD: de alternatieve sleutel van Onderdeel. In je app is dat het Kraken-Id.

Waarom nu. Een gescande code opzoeken is wat het scherm straks het vaakst vraagt. Het is de eerste echte vraag aan je database.

Wat je doet.

- Een interface en een repository tegen je DbContext, meteen asynchroon, zoals in les 05.
- De controller krijgt de repository via zijn constructor; de registratie staat in Program.cs.
- Endpoints: alle onderdelen van dat type, één op Id, en één op Kraken-Id.
- Logging: elk verzoek, en elke 404.

Wat je niet doet. Toevoegen, wijzigen of verwijderen. Je demo krijgt haar gegevens via de startgegevens.

Klaar wanneer Scalar een onderdeel vindt op zijn Kraken-Id, en een onbestaande code 404 geeft.

Valkuil.

- Breek de keten van async nergens door op een asynchrone oproep te wachten alsof ze synchroon is.
- Het Kraken-Id moet uniek zijn. Laat de database dat zelf bewaken: zoek in de Fluent API hoe je een kolom uniek maakt.

Fase 5 · De zes types

Lees eerst: Programming Advanced les 06; leerboek 39. **Uren:** 7 tot 12.

Uit je ERD: de zes types, hun enumeraties, en de overerving.

Waarom nu. De controle vergelijkt onderdelen van verschillende types. Ze moeten er allemaal zijn voor je regels kan maken.

Wat je doet.

- De vijf andere types, elk met de eigenschappen uit fase 2. Ze erven allemaal van Onderdeel.
- De enumeraties uit je ERD als enum.
- Elk type vastleggen met de Fluent API, met de overerving zoals je in fase 2 besliste.
- Van elk type een paar onderdelen als startgegevens, en een migratie: *Je database vullen*.
- Opzoeken op Kraken-Id werkt voor elk type.

Wat je niet doet. De onderdelen van je twee computers volledig invoeren. Dat is fase 6.

Klaar wanneer elk van de zes types op te zoeken is op Kraken-Id, met al zijn eigen gegevens in de JSON.

Valkuil.

- Geef je een onderdeel terug als Onderdeel, dan schrijft de JSON enkel Id, Kraken-Id, Merk en Naam uit. Het begrip om op te zoeken: polymorfe serialisatie. Het hoofdmodel V1 waarschuwde er al voor.
- Een enum verschijnt in je JSON als getal. Wil je tekst, zoek dan op hoe System.Text.Json een enum als tekst schrijft.
- Verandert je keuze voor de overerving hier toch, dan mag je je database en je migraties opnieuw opbouwen: er staan nog enkel startgegevens in. De oefeningen van hoofdstuk 06 doen het ook zo.

Fase 6 · Conflictregels en startgegevens

Lees eerst: Programming Advanced les 06, de relaties; het ontwerp, sectie 5, bij Conflicten. **Uren:** 6 tot 10.

Uit je ERD: niets. Je ERD heeft geen regels: Conflict is daar een conflict dat op een computer gevonden werd. Een regel zegt wat een conflict ís.

Waarom nu. De controle in fase 7 heeft regels nodig om mee te werken. Je legt hier vast wat een conflict is, nog niet hoe je het opspoort.

Wat je doet.

1. **Beslis hoeveel van een regel gegevens zijn, en hoeveel code.** Twee uitersten:

Aanpak	Wat in de database staat	Wat het kost en oplevert
Alles als gegevens	de regel en zijn voorwaarden	Een nieuwe regel kan zonder programmeur. Je controle moet dan voorwaarden kunnen lezen en vergelijken: het moeilijkste stuk van het project.
Gegevens en code	de regel zelf: zijn code, ernst, uitleg, oplossing, en of hij actief is	Wat de regel nagaat, staat per regel in je controle. Sneller gebouwd; een nieuwe regel vraagt een programmeur.

Het ontwerp kiest de eerste. Voor 17 januari is de tweede de veilige keuze, en ze laat de weg naar de eerste open. Jij beslist.

2. **Maak het model van een regel.** Heeft een regel delen in een andere tabel, zoals voorwaarden of de types die hij raakt, leg die relaties dan vast met de Fluent API zoals in les 06, met wat er gebeurt bij verwijderen.
3. **De ernst.** De alpha kent drie waarden: InOrde, Opmerking en NietInOrde. Gebruik je dezelfde namen, dan hoeft de koppeling in fase 8 niets om te zetten.

4. **Startgegevens:** de onderdelen van je twee computers, een paar alternatieven die wél passen, en je demoregels met hun uitleg en oplossing. Hoe: *Je database vullen*.
5. **Een migratie.**

Wat je niet doet. De controle zelf. Ook geen regels die niet in je demo zitten.

Klaar wanneer alle onderdelen van beide computers en al je demoregels in pgAdmin staan, met hun ernst, uitleg en oplossing.

Valkuil.

- Loopt Add-Migration of Update-Database vast, kijk dan in *Je database vullen*, bij *Als het misloopt*.
- De alternatieven maken je demo overtuigend: vervang je het foute onderdeel, dan wordt de regel groen.

Fase 7 · De controle

Lees eerst: niets nieuws. Les 03 voor de registratie, les 05 en 06 voor het ophalen; het ontwerp, sectie 6 en 7.
Uren: 8 tot 12.

Waarom nu. Alles eronder staat. Wat nog ontbreekt, is het stuk dat de vraag beantwoordt: gaan deze onderdelen samen?

Wat je doet.

1. **Beantwoord eerst drie vragen, op papier.** Wat moet de controle teruggeven, opdat het scherm per onderdeel een kleur kan tonen, en per conflict de onderdelen, de uitleg en de oplossing? Een conflict met twee voorwaarden, en maar één onderdeel dat aan allebei voldoet: vuurt het? En een regel over iets dat ontbreekt, zoals losse NZXT-ventilatoren zonder controller: waaraan herkent je controle dat iets er níet is?
2. **De controle als een gewone klasse in Services**, met een interface, geregistreerd zoals in les 03. Ze krijgt onderdelen en regels binnen, en geeft een resultaat terug. Geen database, geen HTTP.
3. **Een endpoint dat een bouworder aanneemt**, een lijst Kraken-Id's, en het resultaat teruggeeft. Er staat geen tabel achter: denk na over welk verb erbij past.
4. **De controller haalt de onderdelen en de regels op** via je repositories, en geeft ze aan de controle.
5. **Toets elke computer** tegen de verwachte uitkomst uit fase 2.

Wat je niet doet.

- De controle in een controller of een repository schrijven.
- De controle zelf de database laten aanspreken.
- Een kleur teruggeven. Je API geeft een ernst; het scherm maakt er een kleur van.

Klaar wanneer beide computers in Scalar precies de verwachte conflicten geven, met ernst, onderdelen, uitleg en oplossing. Met het alternatief in de plaats van het foute onderdeel, verdwijnt het rood.

Valkuil.

- Een bouworder kan hetzelfde Kraken-Id meer dan eens bevatten: twee latjes geheugen, drie ventilatoren. Je regels over aantallen hangen daarvan af.
- Een code die je database niet kent: weiger je de hele bouworder, of meld je dat ene onderdeel? Beslis het vooraf.
- Gerelateerde gegevens komen niet vanzelf mee. Include zie je pas in les 08; tot dan haal je ze in twee stappen op, met wat les 05 je leerde.

Fase 8 · De demo

Lees eerst: de LEESMIJ in de map Alpha van dit pakket. **Uren:** 5 tot 8.

Waarom nu. Een demo die je niet geoefend hebt, toont vooral wat er misloopt.

Wat je doet.

1. **Vraag de koppeling aan, meteen na fase 7.** Stuur me uit Scalar het antwoord van je controle-endpoint voor één computer, het antwoord van je endpoint op Kraken-Id, en het adres waarop je API draait: dat staat in launchSettings.json. Ik schrijf de koppeling in de alpha. Ze zoekt dan een gescande code op via je API, en vraagt de controle aan je controle-endpoint. De verzonnen artikels en de nagebootste controle van de alpha verdwijnen.
2. **Start je API en de alpha samen, en scan je twee computers.** Het scherm moet exact tonen wat Scalar toont: dezelfde ernst, dezelfde onderdelen, dezelfde uitleg en oplossing. Vervang een fout onderdeel door zijn alternatief: op het scherm wordt het groen.
3. **Een demoscript van ongeveer tien minuten:** het probleem op de werkvloer, het scherm met je twee computers, wat erachter zit in Scalar en pgAdmin, en wat er na de demo komt.
4. **Schermafbeeldingen van elke stap**, als plan B voor als iets niet start.
5. **Een generale repetitie** op de laptop waarmee je toont, na een herstart, met Docker, je API en de alpha gestart.

Wat je niet doet. Op de laatste dag nog iets toevoegen. Na de generale repetitie verandert er niets meer.

Klaar wanneer een volledige repetitie lukt van begin tot eind, en het scherm voor beide computers toont wat je API teruggeeft.

Valkuil.

- Je twee computers zijn gekozen om te slagen. Laat iemand een derde bouworder kiezen met onderdelen uit je startgegevens, en kijk of je controle ook die juist beoordeelt.
- Toon geen klant- of ordergegevens: enkel onderdelen.
- Je API en de alpha moeten tegelijk draaien. Visual Studio laat je meerdere startprojecten kiezen. Regel dat voor de repetitie, niet op de dag van de demo.

Deel 2 — Na de demo

Na de demo verbouw je onder de controle, volgens de lessen. De controle zelf blijft staan: ze krijgt nog altijd onderdelen en regels binnen, en geeft een resultaat terug. Dit deel loopt van februari tot mei, en hier remt de school, niet je eigen tijd.

Fase	Les eerst	Uren
9 · Generieke repository en Unit of Work	PA 07	5–6
10 · Gerelateerde gegevens	PA 08	4–5
11 · DTO's en Mapster	PA 09	6–8
12 · Tests rond de controle	ND 08	8–12
13 · Identity, gebruikers en rollen	ND 06 en 07	8–12
14 · Het scherm in je eigen project	ND 09 en 10	8–12
15 · Keuzelijsten en rechten	ND 11	6–10
Na de demo		**45–65**

PA is Programming Advanced, ND is .NET Development. De lessen van .NET Development geven Identity (les 06 en 07) vóór de tests (les 08). Volg je de lessen strikt, dan doe je fase 13 voor fase 12. Beslis het voor deel 2 begint.

Fase 9 · Generieke repository en Unit of Work

Lees eerst: Programming Advanced les 07; leerboek 40. **Uren:** 5 tot 6.

Waarom nu. Je hebt een handvol repositories, en ze doen grotendeels hetzelfde. Deze fase haalt die herhaling eruit.

Wat je doet.

- Een generieke repository die de gewone bewerkingen één keer afhandelt.
- Specifieke repositories die ervan erven, enkel waar een model iets nodig heeft dat de generieke niet biedt, zoals opzoeken op Kraken-Id.
- Een Unit of Work die de repositories beheert, en één keer centraal bewaart.
- Je controllers krijgen de Unit of Work in plaats van losse repositories.

Wat je niet doet. Elke repository specifiek maken. En je controle aanpassen: die verandert hier niet.

Klaar wanneer je controllers nog maar één ding opvragen in hun constructor, er nergens dubbele opzoekcode staat, en je twee computers in Scalar nog altijd hetzelfde antwoord geven.

Valkuil. De les haalt het bewaren uit de repositories en zet het in de Unit of Work. Dat voelt onaf. Het is precies wat ervoor zorgt dat een bewerking over meerdere tabellen in één keer slaagt of in één keer mislukt.

Fase 10 · Gerelateerde gegevens

Lees eerst: Programming Advanced les 08; leerboek 41. **Uren:** 4 tot 5.

Waarom nu. Tot hier haalde je de delen van een regel in twee stappen op. Include doet het in één vraag aan de database.

Wat je doet.

- De repositories die gerelateerde gegevens nodig hebben, laten alles in één keer ophalen met Include.
- Dat gebeurt per model, in de specifieke repository. De generieke repository weet niets van relaties.

Wat je niet doet. Dapper. De les toont het, maar je moet het enkel kunnen uitleggen. De tijdelijke oplossing voor eindeloze lussen die de les geeft, blijft tijdelijk: in fase 11 verdwijnt ze.

Klaar wanneer je terminal bij één controle geen reeks losse vragen aan de database meer toont.

Valkuil. Halverwege van manier wisselen om gegevens op te halen. Blijf bij Entity Framework.

Fase 11 · DTO's en Mapster

Lees eerst: Programming Advanced les 09; leerboek 33 en 42. **Uren:** 6 tot 8.

Waarom nu. Je stuurt nu je eigen modellen naar buiten. Dat geeft eindeloze lussen, toont velden die niemand buiten je API nodig heeft, en elke wijziging aan een model breekt wat ervan afhangt.

Wat je doet.

- Aparte DTO's voor wat je terugstuurt en voor wat je ontvangt.
- De validatie verhuist naar de DTO's die binnenkomen.
- Mapster installeren en de omzetting instellen, zodat je controllers kort blijven.
- De tijdelijke oplossing voor lussen uit fase 10 weghalen.

Wat je niet doet. Een model in een DTO zetten. Een DTO mag naar een andere DTO verwijzen, nooit naar een model.

Klaar wanneer elk endpoint een DTO ontvangt en teruggeeft, en de tijdelijke oplossing uit Program.cs verdwenen is.

Valkuil.

- Veel voorbeelden op internet gebruiken AutoMapper: een andere library, met een andere schrijfwijze en een betalende licentie. Meng ze niet.
- De alpha leest je JSON. Verandert de vorm van je antwoord, dan moet de koppeling mee. Hou de namen gelijk waar het kan; wat gelijk heet, hoef je ook in Mapster niet in te stellen.

Fase 12 · Tests rond de controle

Lees eerst: .NET Development les 08. **Uren:** 8 tot 12.

Waarom nu. Je controle werkt voor de gevallen die je zelf probeerde. Deze fase maakt daar zekerheid van. De les legt ook uit waarom juist een service testbaar is, en een controller of een repository niet.

Wat je doet.

- Het testproject inrichten met NUnit en Moq.
- Tests rond je controle. Je verwachte uitkomsten uit fase 2 zijn je eerste tests. De rest leid je af uit je incompatibiliteitenlijst: een lege bouworder, een regel die vuurt, een regel die net níet vuurt, een regel die niet actief is, een onderdeel in meer dan één conflict.

Wat je niet doet. Tests schrijven die bevestigen wat je controle toevallig doet. Je test wat er hoort te gebeuren, en dat haal je uit je lijst, niet uit je code.

Klaar wanneer je tests groen zijn, met minstens één geval dat je in fase 7 niet met de hand probeerde.

Valkuil. Vind je een fout, pas dan niet meteen je controle aan. Schrijf eerst de test die faalt, en herstel daarna. Anders weet je niet of het echt opgelost is.

Fase 13 · Identity, gebruikers en rollen

Lees eerst: .NET Development les 06, daarna les 07. **Uren:** 8 tot 12.

Waarom nu. Niet iedereen mag de regels wijzigen. Hier beslis je wie op de werkvloer wat mag.

Wat je doet.

- De packages van Identity, een DbContext die van de Identity-variant erft, en een eigen gebruikersmodel.
- De registratie in Program.cs, en een migratie.
- Rollen, en je endpoints afschermen.

Wat je niet doet. Wachtwoorden in je code laten staan. De les maakt een beheerder aan via startgegevens, en waarschuwt er zelf voor.

Klaar wanneer je kan aanmelden, een token krijgt, en een afgeschermd endpoint je zonder token weigert.

Valkuil.

- De opzet van les 06 bevat instellingen die een fout in .NET 8 omzeilen. Op .NET 10 heb je ze niet nodig: neem ze niet blind over.
- Scalar heeft een eigen veld voor je token. De les toont het in Swagger; het idee is hetzelfde.

Fase 14 · Het scherm in je eigen project

Lees eerst: .NET Development les 09, daarna les 10. **Uren:** 8 tot 12.

Waarom nu. Alles eronder werkt en is getest. Nu bouw je het scherm zelf, in je eigen project. De alpha gaat met pensioen.

Wat je doet.

- MVC toevoegen aan hetzelfde project. Les 09 zet twee manieren naast elkaar: de alpha gebruikte de eerste, een scherm dat een API aanspreekt. Je eigen project neemt de tweede: scherm en API samen in één applicatie.
- ViewModels voor wat het scherm nodig heeft, in een eigen map, en de views zelf.
- Het scherm toont wat de alpha toonde: per onderdeel groen, oranje of rood, en per conflict de onderdelen, de uitleg en de oplossing.

Wat je niet doet. De code van de alpha overnemen. Neem de indeling en de feedback van je team mee, niet de code. En de kleur blijft een beslissing van het scherm: je API geeft een ernst.

Klaar wanneer je een reeks codes scant, en het scherm hetzelfde toont als de alpha in je demo.

Valkuil. Een ViewModel is geen DTO, ook al lijkt het erop. Het dient één scherm. Hergebruik je DTO's er niet voor.

Fase 15 · Keuzelijsten en rechten

Lees eerst: .NET Development les 11. **Uren:** 6 tot 10.

Waarom nu. De afwerking: het beheer van de regels heeft keuzelijsten nodig, en de rechten uit fase 13 moeten ook in het scherm zichtbaar zijn.

Wat je doet.

- Keuzelijsten in je formulieren.
- De schermen om aan te melden en gebruikers te beheren.
- Instellen waar iemand terechtkomt die een pagina opent waar hij geen rechten op heeft.

Wat je niet doet. Vertrouwen op een verborgen knop. Een knop die je niet ziet, is nog altijd een adres dat iemand kan intypen.

Klaar wanneer iemand zonder beheerrechten kan controleren, maar geen regels kan wijzigen, en die knoppen ook niet ziet.

Valkuil. Rolnamen moeten exact overeenkomen met wat in de database staat, hoofdletters inbegrepen. Een typfout geeft geen foutmelding, enkel een gebruiker die onterecht geen toegang krijgt.

Daarna

- De drie andere types: videokaart, voeding en opslaggeheugen. Elk type erbij is een migratie, geen herbouw.
- Alle regels uit je analyse en je eigen lijst.
- De onderdelen vullen met de scraper: zie de scrapy-handleiding in het Claude-project.
- De open vragen uit het ontwerp: hoe een bouworder binnenkomt, wie de bron van de regels is, en of rood blokkeert.

De regels waaronder dit plan geschreven is

Uit de projectafspraken, §5:

Geen code schrijven. Geen expliciete voorbeelden die het antwoord voorzeggen. Geen andere cruciale hulpmiddelen die het denkwerk overnemen. Het volledige project moet door mij gemaakt worden. Hulp mag uitsluitend in de vorm van tips en richting geven.

Twee uitzonderingen blijven: de klasse Onderdeel, en de alpha, nu ook met haar koppeling aan je API. Loop je vast in een fase, vraag het dan. Je krijgt een aanwijzing, een vraag terug, of de plek in de les waar het staat.